

工科毕业生小白指南

MCP 服务器生态

看懂 awesome-mcp-servers: 给 AI 装「手和脚」的开放标准

学习对象: github.com/punkpeye/awesome-mcp-servers (2026-10 克隆版 3c30195)

4086

个 MCP 服务器

59

个分类

378

个官方实现 🇺🇸

1

个动手实验 (已验证)

配套实验代码: `mcp_servers\exercise\` | 全部命令实际跑通 ✅

🕒 30 秒速览

MCP (Model Context Protocol, 模型上下文协议) 是一个**开放标准, 让任何 AI 应用用同一种"插口"连接外部工具和数据**——就像 USB-C 统一了充电线。这份清单则是一本收录了 4086 个"USB-C 设备" (MCP 服务器) 的产品目录: 连数据库、控浏览器、查文件、发邮件.....看完本指南, 你将看懂这本目录, 并且**亲手写一个能被真实 Claude 调用的 MCP 服务器**。

本指南回答的 12 个疑问 (即目录)

Q1 MCP 一句话是什么? 解决什么问题?

Q2 没有 MCP 之前人们怎么办? ($M \times N$ 困境)

Q3 它的核心架构: 宿主 / 客户端 / 服务器

Q4 一次工具调用的完整旅程 (含握手)

Q5 服务器能提供的三种能力:
Tools/Resources/Prompts

Q6 两种传输方式: stdio vs Streamable HTTP

Q7 这本目录怎么读? 图例系统详解

Q8 59 类里先玩哪几类? 精选推荐

Q9 和我学过的 LangChain/CrewAI 工具什么关系?

Q10 安全吗? 第三方服务器的风险与防线

Q11 怎么把现成服务器装进 Claude?

Q12 自己写一个难吗? (→ 动手实验)

Part 1 · 它是什么，为什么存在

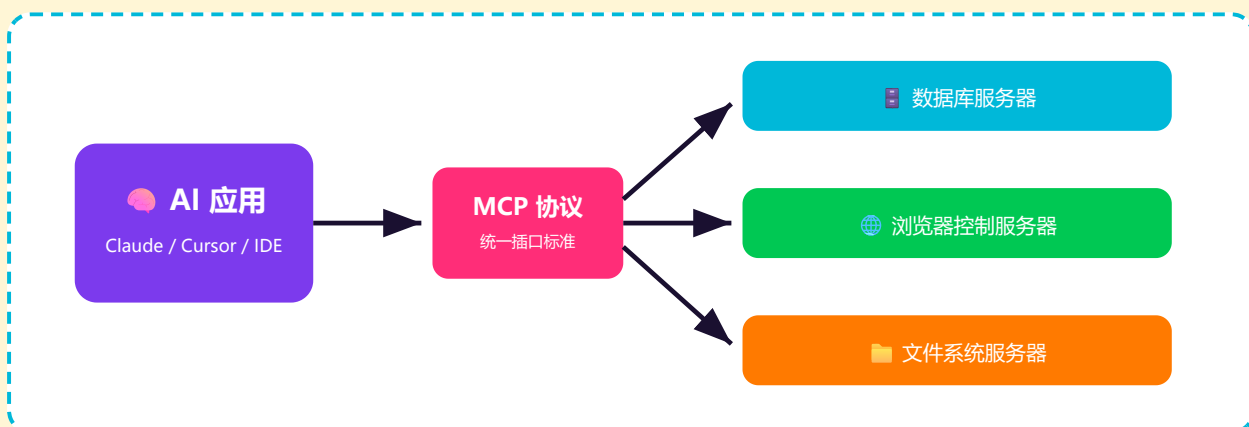
Q1 MCP 一句话是什么？解决什么问题？

一句话：MCP 是 AI 应用连接外部工具与数据源的开放协议——2024 年 11 月由

Anthropic 开源，如今已是事实上的行业标准。

问题很简单：大模型本身只会“说话”，它没有手，不能帮你查文件、点按钮、发消息。要让它干活，就得有人给它造“手脚”。MCP 就是造手脚的统一图纸：按图纸造出来的零件（MCP 服务器），能插到任何认这个标准的 AI 应用（Claude Desktop、Claude Code、Cursor 等）上。

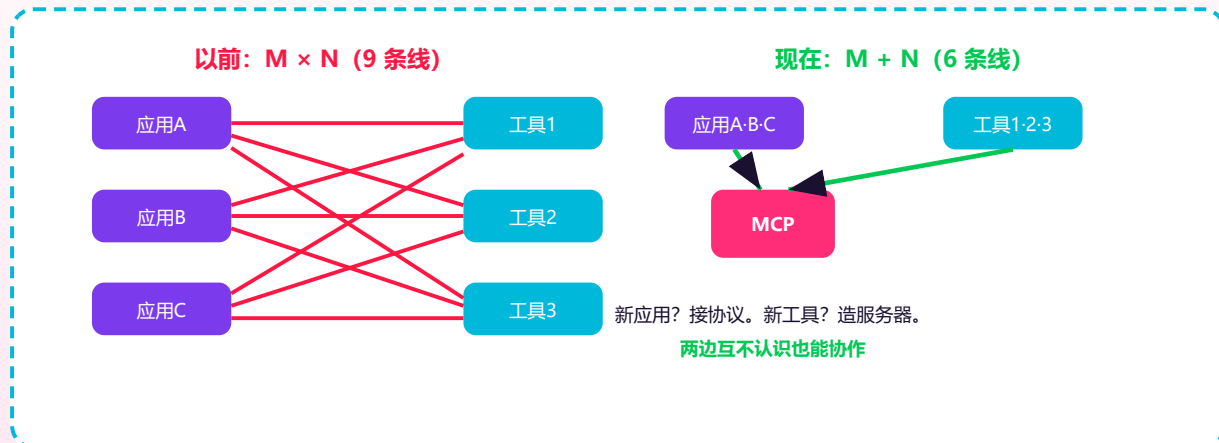
🔑 类比（讲完落回术语）：MCP 之于 AI 工具，就像 USB-C 之于外设——以前每个手机一个专用充电口，现在一根线通吃。注意：MCP 不是一个软件，而是协议（双方约定的通信规则，规定了消息格式和流程）；具体干活的程序叫 MCP 服务器，把它接进 AI 应用的动作叫“安装一个 MCP 服务器”。



这份 awesome-mcp-servers 清单，就是全社区按这个标准造好的“外设”总目录——所以学会读它 ≈ 拥有一个 AI 能力商店。

Q2 没有 MCP 之前，人们怎么做？

2024 年之前的做法：**每个 AI 应用为每个工具单独写一套对接代码**。你有 M 个 AI 应用、N 个工具，就要写 $M \times N$ 份胶水代码。ChatGPT 的插件体系和 Claude 的早期工具互不通用；你自己在 LangChain 里写的工具（还记得 `exp2_make_tool` 吗？），拿到 Cursor 里就完全用不了。

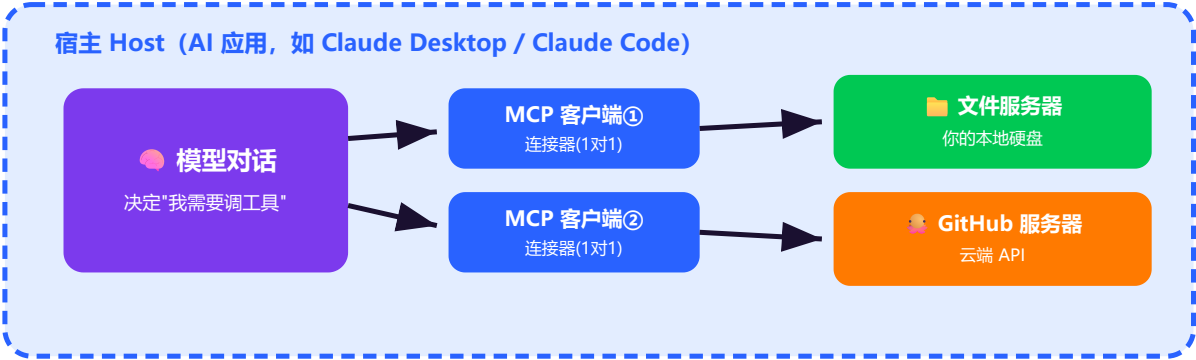


把 $M \times N$ 变成 $M + N$: **每个应用只需实现一次客户端，每个工具只需包成一次服务器**。这就是"协议"的经典价值——和 [[构建流水线与Pass]] 里"统一接口让阶段可替换"是同一个思想，只是这次统一的是整个行业。

Part 2 · 它是怎么工作的

Q3

核心架构：宿主 / 客户端 / 服务器



角色	是什么	本例中
宿主 Host	你想用的 AI 应用本体，内部可同时连多个服务器	Claude Desktop / Claude Code
MCP 客户端	宿主内部的“连接器”，每个服务器对应一个客户端实例，负责收发协议消息	Claude 里的 MCP 连接模块
MCP 服务器	独立进程/服务，把某种能力（文件、API、数据库）按协议暴露出来	目录里的 4086 个项目

关键认知：服务器是独立于宿主的程序——它可以是本地子进程，也可以是云上的服务。底层消息格式是 **JSON-RPC 2.0**（一种“发方法名+参数、收回执”的通用远程调用格式，快递单号式通信）。

Q4 一次工具调用的完整旅程



三步记住: **先握手 (initialize) → 再报价 (tools/list) → 后干活 (tools/call)**。握手时双方交换 **能力 (capabilities)** ——就像开店先互相递营业执照, 客户端从此知道这家服务器会哪些手艺。你会在实验第 3 步亲眼看到这三步的输出。

Q5 服务器能提供的三种能力

能力	一句话定义	谁主导使用	例子（实验里都有）
Tools 工具 🔧	可执行的 动作 ，可能有副作用	模型决定何时调	add(a,b)、查数据库、发邮件
Resources 资源 📄	只读 数据 ，URI 标识，无副作用	应用/用户主动读取	lab://greeting/小明、日志文件
Prompts 提示模板 💬	预制好的 提示词模板	用户手动选用	"/代码评审"一键模板

判断口诀：会“做事”的是 Tool，只是“被看”的是 Resource，给人“点选”的话术模板是 Prompt。目录里绝大多数项目的主打能力都是 Tools。另外客户端也能向服务器反向提供能力（采样 Sampling、根目录 Roots、引导输入 Elicitation），小白阶段知道有这回事即可。

Q6 两种传输方式：stdio vs Streamable HTTP



这正好对应目录里每个条目的 **🏠 Local (本地服务)** 和 **☁️ Cloud (云端服务)** 标记：管本地软件（如控制 Chrome）的是 🏠，调远程 API（如天气、GitHub）的多为 ☁️。本地型更私密（数据不出机器），云端型更省事（不用装环境）。

Part 3 · 读懂这本 4086 项的目录

Q7 目录怎么读？图例系统详解

每个条目都是这个格式（一眼四问：叫什么、什么语言、跑在哪、能干嘛）：

- [box/mcp-server-box-remote](https://github.com/box/mcp-server-box-remote/) 🏠 ☁️ ← 图例标签 -
Access Box content: search, ask questions about files... ← 一句话说明能干什么

标签	含义	选择建议
🏆	官方实现（厂商自家出品）	优先选这个，质量与维护最有保障（共 378 个）
🔄 📄 🛠️ 🦋 #	代码语言：Python / TypeScript / Go / Rust / C# / Java / C++ / Ruby	想改造它就选你会的语言
☁️ / 🏠 / 💻	Cloud 云服务 / Local 本地服务 / 嵌入式	看你在意省事还是隐私
🍏 🖥️ 🐧	支持的系统：macOS / Windows / Linux	咱是 Windows，认准 🖥️

📖 逛目录的姿势：59 个分类按字母排（Aggregators 聚合器 → Version Control 版本控制 → Other 其他）。别从头翻到尾！正确姿势：先想“我想让 AI 会干什么”，再按分类直达。目录还有官方网页版 glama.ai/mcp/servers 可搜索，另有姊妹清单 [awesome-mcp-clients](#)（客户端）和 [awesome-remote-mcp-servers](#)（免安装的托管服务器）。

Q8 59 类里先玩哪几类？

结合你正在学的 agent 技术栈（LangChain / CrewAI / 本地 Ollama），推荐这条路线：

优先级	分类	能干什么	代表场景
★★★	File Systems 文件系统	让 AI 安全地读写指定目录	"帮我把这个文件夹里的 PDF 全部改名"
★★★	Browser Automation 浏览器自动化	驱动 Chrome/Playwright 操作网页	"打开某网站帮我查价格填表格"
★★★	Databases 数据库	自然语言查 SQL 库	"上个季度销量前十的产品是哪些？"
★★	Version Control 版本控制	操作 git / GitHub	"把这周的提交整理成周报"
★★	Knowledge & Memory 知识记忆	给 AI 挂长期记忆库（有 Obsidian 相关服务器！）	"记住我的项目背景，下次接着聊"
★★	Aggregators 聚合器	一个服务器转发到 N 个应用	一个入口连所有 SaaS
★	Command Line / OS Automation	执行命令、控制操作系统	定时任务、批量操作（谨慎授权）

装 3~5 个精选 > 装 30 个囤货：每个服务器都会往模型上下文里塞工具说明书，装太多反而让模型选择困难、上下文变贵。

Part 4 · 与你的知识网对接 & 安全

Q9 和 LangChain / CrewAI 的工具是什么关系?

框架内工具 (你写过: `exp2_make_tool`)

LangChain 工具

CrewAI 工具

= Python 对象, 只在本框架内可用, 换框架就作废

MCP 服务器 (协议化工具)

一份实现: Claude / Cursor / LangChain 都能接

= 独立进程 + 标准协议, 跨应用、跨语言、跨框架复用

关系不是替代, 是分层: LangChain/CrewAI 现在都能把 MCP 服务器当工具源接入——框架管"怎么思考和编排" (对应你学过的 agent 循环), MCP 管"工具怎么标准化地被找到和调用"。你在 langgraph 练习里做的 [[质检Gate与自我纠错循环]], Gate 调用的检查器以后就可以是一个 MCP 服务器, 多个项目共用。

Q10 安全吗? 第三方服务器的风险与防线

先把最要命的说清楚: MCP 服务器是以你的身份在跑的程序。 它能读你的文件、执行命令——装一个来路不明的服务器 ≈ **给陌生人发了把自家钥匙。**

风险	是什么	防线
权限过大	服务器能做你用户能做的一切事	只给最小目录/最小权限 (如文件服务器只开放指定文件夹)
提示注入	恶意网页/文档里藏指令, 诱导模型滥用工具 ("工具投毒")	敏感操作保持人工确认, 别全放自动批准
供应链投毒	第三方服务器本身带恶意代码或被植入	优先 🏆 官方; 看 star 与维护活跃度; 锁版本
密钥泄露	API key 写进配置、随对话外泄	用环境变量/密钥保险库类服务器 (目录 Security 分类有专门的方案)

这与你知识库里的 [[质检Gate与自我纠错循环]] 一脉相承: **能力越强, 越要在自动流程里设人工检查站。**

Part 5 · 上手：安装与自造

Q11 怎么把现成服务器装进 Claude?

```
# Claude Code 一条命令注册（以官方文件服务器为例）
claude mcp add filesystem -- npx
-y @modelcontextprotocol/server-filesystem D:\我的资料 # Claude Desktop 则编辑配置文件
%APPDATA%\Claude\claude_desktop_config.json: { "mcpServers": { "filesystem":
{ "command": "npx", "args": ["-y", "@modelcontextprotocol/server-filesystem", "D:\我的资料"] } } }
```

✓ **验证方式**：Claude Code 用 `claude mcp list`，看到 ✓ **Connected** 即成功；Claude Desktop 重启后输入框旁出现工具图标。

⚠ **实验中真实踩到的三个坑（都会表现为 "Connection closed"）**：

- ① **解释器不对**：写 "python" 会用系统 Python（没装依赖）→ 用 `venv` 的绝对路径；
- ② **路径反斜杠被 shell 吃掉**：`bash` 里必须给整个路径加单引号 `'F:\...'`；
- ③ **SDK 大版本改名**：`mcp 2.x` 已把 `FastMCP` 改名 `MCPServer`，照抄老教程会 `ModuleNotFoundError`。
排查通法：先在终端手动运行服务器命令，看它能不能独立启动——启动报错会直接打印出来。

Q12 自己写一个难吗？——不难，往下看实验

官方 SDK（Python 包名就叫 `mcp`）把写服务器压缩到三步：**建实例** → **挂函数** → **run**。函数签名和文档字符串会被自动转成“工具说明书”喂给模型——这就是框架的 **装饰器** 魔法（一个“注册器”，函数定义时顺手登记进目录）。

```
from mcp.server.mcpserver import MCPServer
mcp = MCPServer("awesome-mcp-lab")
@mcp.tool()
def add(a: int, b: int) -> int: """把两个整数相加。""" ← 这行会变成给模型看的说明书！
return a + b
mcp.run() # stdio 传输：被客户端当子进程启动
```

接下来进入动手实验——所有命令都已在你的机器上实跑验证。



动手实验：给你的 AI 装上第一个自造 MCP 服务器 (已验证)

实验目标：用官方 Python SDK 写一个 MCP 服务器（2 个工具 + 1 个资源），先用最小客户端验证协议三步曲，再注册进 Claude Code 让真实模型调用。**成就感时刻：**第 5 步你会看到 Claude 亲口报出你的工具算出的答案。

实验材料已放在 `mcp_servers\exercise\`（`server.py` / `client.py`），可跟做，也可从零敲一遍加深理解。

第 1 步

建环境：虚拟环境 + 安装官方 SDK 目的：隔离依赖

```
cd F:\reminder\mcp_servers python -m venv .venv .venv\Scripts\python -m pip install "mcp[cli]"
```

目的：获得官方 SDK（含 CLI 调试器）。**实现：**装好 mcp 2.2.0。

✓ 预期结果：安装结束无报错；`.venv\Scripts\python -c "import mcp; print(mcp.__version__)"` 输出 **2.2.0**（或更高）。

第 2 步

写服务器 `server.py`：2 个工具 + 1 个资源 目的：造出"USB-C 设备"

完整代码见 `exercise\server.py`（Q12 已展示核心）。三个能力：**add**（加法工具）、**count_servers**（统计真实 awesome 清单：条目数/分类数/官方数）、**greeting 资源**（体验 Resource 和 Tool 的差别）。**实现：**一个约 40 行的可用 MCP 服务器。

✓ 预期结果：文件保存后无需单独测试——下一步客户端会替你检验（服务器是被动等连接的）。

第 3 步

写最小客户端 client.py 并运行：亲眼看协议三步曲

目的：理解握手→列表→调用

```
cd F:\reminder\mcp_servers\exercise ..\venv\Scripts\python client.py
```

实现：client.py 把 server.py 当子进程启动，走完 initialize → tools/list → tools/call → read_resource。

✓ 预期结果（实跑输出）：

[握手成功] 服务器：awesome-mcp-lab

[协议版本] 2025-11-25

[工具清单] ['add', 'count_servers']

[调用 add(20, 22)] -> 42

[调用 count_servers()] -> {"总条目数": 4086, "分类数": 59, "官方实现数": 378}

[读取资源] -> 你好 小明！这是从 MCP 资源 lab://greeting/小明 读到的内容。

⚠ 若报 **Connection closed**：九成是 client.py 里用了系统 python 启动子进程——改用 sys.executable（见代码注释）。

第 4 步

注册进 Claude Code

目的：接上真实宿主

```
claude mcp add awesome-lab --scope local -- ^ 'F:\reminder\mcp_servers\venv\Scripts\python.exe' ^  
'F:\reminder\mcp_servers\exercise\server.py' claude mcp list
```

实现：把服务器登记进 Claude Code 的项目级配置。**注意**：路径必须用**绝对路径**；bash 里单引号包住反斜杠路径（Q11 坑②）。

✓ 预期结果：claude mcp list 末尾出现 **awesome-lab: ... - ✓ Connected**。

第 5 步

端到端：让 Claude 真的调用你的工具 成就感时刻

```
claude -p "调用 awesome-lab 的 add 工具计算 20+22，再调用 count_servers，一行报告结果" --allowedTools "mcp_awesome-lab__add" "mcp_awesome-lab__count_servers"
```

实现：--allowedTools 免去了交互确认，让无头模式直接放行你的两个工具。

✅ 预期结果（本机实跑原话）：

"add 结果：20+22 = 42；awesome-mcp-servers 清单统计：共 4086 条目、59 个分类、378 个官方实现。"

——数字是你的代码算的，措辞是模型组织的：这就是 MCP 全链路打通的铁证。

🔪 收尾与进阶

```
claude mcp remove awesome-lab -s local # 不想留就删掉注册
```

改造练习（由易到难）：① 给服务器加一个 search_category(keyword) 工具，按关键词搜清单分类；② 把 greeting 资源改成读你自己的一篇笔记；③ 用 npx @modelcontextprotocol/inspector 打开官方调试器，可视化看消息流；④ 读目录 Aggregators 分类，思考"一个服务器如何转发到多个应用"。



这个项目给我什么

能力清单

你能做的事	怎么做
🔧 给 Claude 装新技能（查库、控浏览器、读文件...）	从目录挑 🏆 官方服务器 → <code>claude mcp add</code> 一条命令
🔗 把你写过的工具升级为跨应用通用版	LangChain 工具逻辑挪进 MCPServer，一次编写处处可用
🧠 给多项目 agent 挂共享记忆/知识库	Knowledge & Memory 分类（含 Obsidian 服务器）
🏢 为团队做内部能力网关	把公司内部 API 包成 MCP 服务器，全员 Claude 可用
🔍 精准淘服务器	看懂图例（🏆/语言/🏠/系统），配合 <code>glama.ai</code> 搜索
🛡️ 安全地评估第三方服务器	按 Q10 防线清单过一遍再装



实验中踩过的坑（比成功更值钱）

#	坑	解法
1	客户端用系统 python 启动子进程 → 没装依赖 → Connection closed	StdioServerParameters 用 <code>sys.executable</code> 或 <code>venv</code> 绝对路径
2	mcp 2.x 把 FastMCP 改名 MCPServer，老教程代码直接报错	用 <code>from mcp.server.mcpserver import MCPServer</code> ；属性也改 <code>snake_case</code> (<code>server_info</code>)
3	bash 里路径反斜杠被吞 → 注册的命令面目全非	整条路径用单引号包住

📖 概念入库 & 与知识库的连接

本项目新增概念： **MCP协议** 、 **MCP服务器** 、 **stdio传输** 、 **JSON-RPC** 、 **Tools/Resources/Prompts**

。

与已有概念关联：[[构建流水线与Pass]]（协议=行业级统一接口）、[[质检Gate与自我纠错循环]]（工具越强越要Gate）、[[确定性脚本]]（服务器行为可预测才值得信任）。

下一步衔接：正在学的 LangChain/LangGraph/CrewAI 都能接 MCP 服务器——学完它们回头把实验工具接进 agent 循环，就是一次漂亮的跨项目综合练习。

生成于 2026-10-01 · awesome-mcp-servers @ 3c30195 · 实验环境 Python 3.14.7 / mcp SDK 2.2.0 / Claude Code 实测通过